# Introduction

You are reading the reference documentation for the Bamboo Specs. The Bamboo Specs allows you to define Bamboo configuration as code, and have corresponding plans/deployments created or updated automatically in Bamboo. Read more about the Bamboo Specs here.

If you are looking for detailed information about specific methods or classes, see Bamboo Specs JavaDocs and Bamboo Specs YAML references.

Storing Bamboo Specs in a repository allows you to keep your project configuration together with the code and automatically publish any code changes. It also gives you access to history of plan specification, and makes it easy to revert to a particular moment in time. In this documentation Repository stored specs will be referenced as RSS.

## YAML

If this is the first time you are reading this documentation, and you plan to use YAML Specs, you may benefit from starting your read from Understanding YAML in Specs.

## Version information

This reference documentation is based on Bamboo Specs library version **10.0.2**, which is intended for Bamboo version **10.0.2** and higher.

> Starting from Bamboo 6.9, we introduce a new version of YAML, version 2.
>
> The version 2 is **not backwards compatible** but all your plans written in YAML version 1 will still work in future versions of Bamboo.

```
---
version: 2
# ...
```

**Feedback**

In case you found a bug, would like to suggest an improvement or just would like to share your impression about this documentation, please don't hesitate to report it here.

## Bamboo Specs location

**YAML**

For YAML Bamboo Specs, you must have your `bamboo.yaml` file at the directory `bamboo-specs/` in the root of your repository.

`[your_repo]/bamboo-specs/bamboo.yaml`

**JAVA**

For Java Bamboo Specs, you must have your `pom.xml` at the directory `bamboo-specs/` in the root of your repository.

`[your_repo]/bamboo-specs/pom.xml`

You can find the tutorial on how to start with Java Specs in our documentation

# Project

> It is not possible to create a project with YAML Specs.
>
> Create a project in Bamboo before starting writing your YAML files.

A project is a collection of plans. Projects enable you to easily group and identify plans which are logically related to each other. They are especially useful when generating reports across multiple plans.

A project:

- has one or more plans
- provides reporting (for example, using the wallboard) across all plans in the project
- manages permissions for plans
- provides links to other applications
- can store its own build resources

You can create a new project independently or during plan creation.

## Project level build resources

You can manage variables on the project level.

> This feature is available for Bamboo DC licenses only.

Build resources, such as repositories or shared credentials, can be created on the project level while creating or updating the project.

To add, edit, or remove project resources, you must publish a project entity.

# Plan

```yaml
---
version: 2
plan:
  project-key: ROCKET
  name: Launch Rocket
  key: LAUNCH

stages:
  - First Build Stage:
      jobs:
        - Build Ship Server

Build Ship Server:
  tasks:
    - script:
        - echo 'Hello world'
```

A **plan** defines everything about your continuous integration build process in Bamboo.

A plan:

- has a single stage, by default, but can be used to group jobs into multiple stages
- processes a series of one or more stages that are run sequentially using the same repository
- specifies the default repository
- specifies how the build is triggered, and the triggering dependencies between the plan and other plans in the project
- specifies notifications of build results
- specifies who has permission to view and configure the plan and its jobs
- provides for the definition of plan variables

Every plan belongs to a project. You can create a new project when publishing a plan, but you can't update an existing project or its resources from the plan configuration.

# Stage

Stages are defined by their name.

```yaml
---
version: 2
stages:
  - First Stage:
      - Build Binaries
      # ...
  - Second Stage:
      - Testing
      # ...
```

Each stage will contain a set of jobs that may be executed in parallel.

```yaml
---
version: 2
stages:
  - First Stage:
      jobs:
        - Job A # These jobs will be defined separatedly, on the root of the document
        - Job B
```

You can also define manual and final stages.

```yaml
---
version: 2
stages:
  - Build the website:
      jobs:
        - Y
  - Wait for confirmation:
      manual: true # Will await for execution by user
      jobs:
        - A
        - B
  - Clean afterwards:
      final: true # Will be executed even if previous results fail
      jobs:
        - C
        - D
```

**Stages** group **jobs** to individual steps within a plan's build process. For example, you may have a build process that comprises of a compilation step, followed by several test steps.

You can create separate Bamboo stages to represent each of these steps.

A stage:

- shall have at least one job
- processes its jobs in parallel, on multiple agents (if available)
- must successfully complete all its jobs before the next stage in the plan can be processed
- may produce artifacts that can be made available for use by a subsequent stage

Each plan should contain at least one stage.

---

# Job

---

Jobs are defined in the root of the YAML file.

```
---
version: 2
First Job:
  # ...
Second Job:
  # ...
```

They can be named with any alphanumeric character, and include spaces, hyphens, commas, or dots.

```
---
version: 2
1:
  # ...
Special-rocket-job:
  # ...
```

Multiple stages can use the same job definition.

```
---
version: 2
stages:
  - Build the first engine
      jobs:
        - Special engine configuration
  - Second engine
      jobs:
        - Special engine configuration
Special engine configuration:
  # ...
```

A **job** is a single build unit within a plan and is made up of one or more tasks.

Jobs in a stage will be run in parallel, if enough Bamboo agents are available and your license size is adequate.

A job:

- processes a series of one or more tasks that are run sequentially on the same agent
- controls the order in which tasks are performed
- collects the requirements of individual tasks in the job, so that these requirements can be matched with agent capabilities
- defines the artifacts that the build will produce
- can use artifacts produced in a previous stage

Each plan should contain at least one job.

# Tasks

Tasks are defined inside jobs.

```yaml
---
version: 2
Job number 1:
  tasks:
    - script: echo 'Hello world'
```

You can define multiple tasks.

```yaml
---
version: 2
# ...
Integration tests:
  tasks:
    - clean
    - inject-variables: globals.txt
    - script:
        interpreter: /bin/sh
        scripts: make install
    - script:
        - touch report.xml
```

Final tasks are defined on a separated key, executed after all the tasks, and accept the same elements as the tasks.

```yaml
---
version: 2
# ...
Integration tests:
  tasks:
    - clean
    - script:
        - touch report.xml
  final-tasks:
    - test-parser: testng
```

A **task** is a small unit of work, such as source code checkout, or running a script. Tasks are configured within the scope of a job and run on a Bamboo working directory.

Bamboo can have two type of tasks:

| | |
|---|---|
| **Build tasks** | Will run sequentially in the order specified in the job; if a build task fails, all subsequent tasks will not be executed. |
| **Final tasks** | Will run sequentially, once the build tasks have completed; final tasks will always be executed, regardless of whether any build tasks or other final tasks fail; final tasks will be executed even if you stop the build manually. |

A task may make use of an executable if required.

## Task: Artifact Downloader

A task downloading all artifacts produced by plan “PROJECTKEY-PLANKEY” to the working directory.

The task also downloads an artifact `an artifact` to a folder `subdirectory`.

```
---
version: 2
# ...
Default job:
  tasks:
    - artifact-download:
        source-plan: PROJECTKEY-PLANKEY
        # Allows you to configure multiple artifacts from
        artifacts:
           # Downloads all artifacts to the `/` directory
           - destination: /
           # Downloads the artifact called `an artifact` to `subdirectory`
           - name: an artifact
             destination: subdirectory
```

A task downloading all artifacts for a deployment project.

```
---
version: 2
# ...
Environment:
  tasks:
    # The task without specified artifacts will download all artifacts
    - artifact-download
```

The **Artifact Download** task copies Bamboo shared artifacts to a specified folder. This task allows sharing artifacts between different build plans.

For example, you can run acceptance tests on a particular build from a different plan by sharing the same WAR artifact. This will allow both plans to use the artifact without rebuilding each time.

In order to configure the task:

- Select a plan that produces the artifact(s).
- Define the artifacts that you want to download. In Java Specs you will define `DownloadItem` objects, which combine name and destination path. In YAML Specs you will define items inside the `artifacts:` option. You can add as many of those as required.

In order to specify single download request:

- Pick an artifact to download by passing its name.
- Define a path of a folder artifact(s) should be downloaded to. The path can be either absolute or relative to the working directory. If not specified, artifacts are downloaded directly to the working directory.

---

# Task: Clean Working Directory

A task cleaning working directory.

```
---
version: 2
# ...
  tasks:
    # There is no option to be configured
    - clean
```

Clean task with other tasks.

```
---
version: 2
# ...
```

```
  tasks:
    - clean
    - script:
        - touch report.xml
```

The **Clean Working Directory** task cleans the working directory of the job. This task will remove all the files from the working directory when executed.

For example, use this task to enforce a clean build, or to save space on the agent once the build is complete.

---

# Task: Script

Run a script using a default interpreter.

```
---
version: 2
# ...
  tasks:
    # The pipe '|' allows multiline strings in YAML
    - script: |
        #!/bin/bash
        Hello Bamboo
```

Short syntax with default interpreter `SHELL`.

```
---
version: 2
# ...
  tasks:
    - script: path/to/my/script.sh
```

Run a script with an argument, environment variable and in the `rockets` working subdirectory.

```
---
version: 2
# ...
  tasks:
    - script:
        interpreter: /bin/sh
        file: ant-build.sh
        argument: --verbose
        environment: ANT_OPTS=-Xmx700m
        working-dir: rockets
```

The **Script** task allows you to define a script to be run in bash on Linux, or command files on Windows.

You can specify which interpreter to use:

| | |
|---|---|
| **shell (default)** | Interpreter will be chosen based on the *shebang line* of the script. |
| **Windows PowerShell** | Interpreter will be the Windows PowerShell. |
| **/bin/sh or cmd.exe** | Interpreter will be /bin/sh or cmd.exe, chosen based on the operating system. |

In Java Specs, you can define the script inline using the `inlineBody` method. Also, you can load a script from an external file using the `fileFromPath` method.

Additional options you can specify are:

| | |
|---|---|
| **argument** | Specify an argument to pass to the script. Arguments that contain spaces must be quoted. You can include variables (see Bamboo variables). |
| **environment variables** | Additional system environment variables that you want to pass to your build. Existing environment variables are automatically available to the executable. You can also include Bamboo global or build-specific variables (see Bamboo variables) Multiple variables should be separated with spaces. Parameters with spaces must be quoted. |
| **working subdirectory** | Alternative subdirectory, relative to the job's root directory, where Bamboo will run the executable. The root directory contains everything checked out from the job's configured source repository. If you leave this field blank, Bamboo will look for build files in the root directory. This option is useful if your task has a build script in a subdirectory and the executable needs to be run from within that subdirectory. |

# Task: Inject Variables

```yaml
---
version: 2
# ...
  tasks:
    - inject-variables:
        file: folder\file.txt
        scope: RESULT # case insensitive
        namespace: myspace
```

With default values:

```yaml
---
version: 2
# ...
  tasks:
    - inject-variables: tests.txt
    # The default values will be
      # scope: LOCAL
      # namespace: inject
```

The **Inject Variables** task allows you to read the values for variables from a file, and create those variables in your build plan.

The file should use a `key=value` format.

You can set the scope of variables:

| | |
|---|---|
| **LOCAL** | Injected variables are bounded to the job executing. They cease to exist when the job finishes. |
| **RESULT** | Injected variables are persisted and passed into subsequent stages or related deployment releases. |

# Task: Test Results Parser

Mocha test results parser:

```yaml
---
version: 2
# ...
  final-tasks:
```

```
    - test-parser:
        type: mocha
        test-results:
          - mocha-1.json
          - mocha-2.json
```

```
---
version: 2
# ...
  final-tasks:
    # This will parse the default files `**/test-reports/*.xml` for junit
    - test-parser: junit
```

The **Test Results Parser** task in Bamboo parses test data. It may also run tests, using a particular testing framework.

These are the default configurations for the test parsers:

| Test parser | Default reports location |
|---|---|
| JUnit | `**/test-reports/*.xml` |
| NUnit | `**/test-reports/*.xml` |
| TestNG | `**/testng-results.xml` |
| Mocha | `mocha.json` |

You can also use test parsers included in other tasks. For example, within Java build tasks, like Maven, you can specify that test results will be produced.

The separated task for Test parser may be useful for:

- setting up one JUnit Parser task to parse test data for a number of Maven tasks after they all have been executed
- using test parsers for .NET builder tasks in Bamboo - you must configure a test task like NUnit if you want the test results from NAnt builds

#### JUNIT PARSER

Scan JUnit test reports from `build/test/reports` and `target/test/xml-reports` directories.

```
---
version: 2
# ...
  final-tasks:
    - test-parser:
        type: junit
        test-results:
          - build/test/reports
          - target/test/xml-reports
```

| | |
|---|---|
| **Result directories** | List of Ant patterns matching directories and files to be scanned. |
| **Default result directory** | Use the `**/test-reports/*.xml` pattern. |
| **Pick-up test results outside of this build** | Allows the task to scan test result files created before start time of the build. Enabling it may be useful if you don't clean the build working directory and perform incremental build and tests. Enabling will aggregate test results from many builds. |

#### TESTNG PARSER

Scan TestNG reports from default location, and pick test results from previous builds.

```yaml
---
version: 2
# ...
  final-tasks:
    - test-parser:
        type: testng
        ignore-time: true
```

| | |
|---|---|
| **Result directories** | List of Ant patterns matching directories and files to be scanned. |
| **Default result directory** | Use the `**/testng-results/*.xml` pattern. |
| **Pick-up test results outside of this build** | Allows the task to scan test result files created before start time of the build. Enabling it may be useful if you don't clean the build working directory and perform incremental build and tests. Enabling will aggregate test results from many builds. |

#### MOCHA PARSER

Scan Mocha reports from the default location.

```yaml
---
version: 2
# ...
  final-tasks:
    - test-parser:
        type: mocha
```

| | |
|---|---|
| **Result directories** | List of Ant patterns matching directories and files to be scanned. |
| **Default result directory** | Use the `mocha.json` pattern. |
| **Pick-up test results outside of this build** | Allows the task to scan test result files created before start time of the build. Enabling it may be useful if you don't clean the build working directory and perform incremental build and tests. Enabling will aggregate test results from many builds. |

#### NUNIT PARSER

Scan NUnit reports from the default location.

```yaml
---
version: 2
# ...
  final-tasks:
    - test-parser:
        type: nunit
```

| | |
|---|---|
| **Result directories** | List of Ant patterns matching directories and files to be scanned. |
| **Default result directory** | Use the `**/test-reports/*.xml` pattern. |
| **Pick-up test results outside of this build** | Allows the task to scan test result files created before start time of the build. Enabling it may be useful if you don't clean the build working directory and perform incremental build and tests. Enabling will aggregate test results from many builds. |

#### MSTEST PARSER

Scan MSTest reports from the default location.

```yaml
---
version: 2
# ...
  final-tasks:
    - test-parser:
        type: mstest
```

| | |
|---|---|
| **Result directories** | List of Ant patterns matching directories and files to be scanned. |
| **Default result directory** | Use the `**/*.trx` pattern. |
| **Pick-up test results outside of this build** | Allows the task to scan test result files created before start time of the build. Enabling it may be useful if you don't clean the build working directory and perform incremental build and tests. Enabling will aggregate test results from many builds. |

---

# Task: Repository Checkout

If no checkout task specified then every job configuration created in YAML has the repository checkout as its first task. If at least one checkout task is present at job configuration then default repo checkout task should be provided by user.

Provide no parameters and checkout at any time of job execution. Default plan repository (which contains YAML file) will be checked out.

```yaml
---
version: 2
# ...

Build Ship Server:
  tasks:
    - script:
        - echo 'Hello world'
    - checkout
```

Provide linked repository name, destination path and force clean checkout:

```yaml
---
version: 2
# ...
Build Ship Server:
  tasks:
    - script:
        - echo 'Hello world'
    - checkout:
        repository: instructions
        path: my-path
        force-clean-build: false
```

Use checkout task for environment configuration deploy scripts. Simple form allows to specify any linked repository.

```yaml
---
version: 2
#...

QA:
  tasks:
    - checkout: instructions
```

```
    - script:
      - echo 'Release planet'
```

Full format for deployment environment.

```
---
version: 2
# ...

QA:
  tasks:
    - checkout:
      repository: instructions
      path: my-path
      force-clean-build: false
...
```

The **Repository Checkout** task is used to check out a repository for single use in a job. By default, the repositories are checked out to the Bamboo working directory. The task allows the checkout of multiple repositories, but the order in which repositories will be checked out is undefined.

How to configure the task:

- Specify the repository and the path where it will be checked out. In Java, use the `CheckoutItem` object. You can use multiple objects of this type.
- Decide if task should perform a clean checkout. If set, the task will remove contents of each specified folder before checking out the source code.

A repository can be specified by its name or identifier. The repository can be a plan repository or a linked repository.

You can also use the `defaultRepository` option. The `defaultRepository` option will enable checkouts from the first repository that was added or linked to the plan, regardless of its actual definition.

The paths are always relative to the working directory.

---

# Task: Repository Tag

YAML Specs does not support the Repository Tag task.

The **Repository Tag** task enables the creation of new tags in the checked out repository. The task will create a new tag for the last commit in the checkout directory.

For DVCS repositories, the task will push the newly created tag to the remote repository.

To use the task:

- Make sure that a Repository Checkout task is executed beforehand.
- Select a repository:
  - If the task is used in a build plan, you can select the default repository of that plan.
  - If the task is used in a deployment, you can specify the working directory into which the repository was checked out.
  - In either case you can choose a repository using its name;
- Provide the tag name, you may also use Bamboo variables.

---

# Task: Repository Branch

YAML Specs does not support Repository Branch task.

The **Repository Branch** task enables the creation a new branch in the checked out repository. The task will create a new branch from the last commit in the checkout directory. For DVCS repositories, the task will push the newly created branch to the remote.

To use the task:

- make sure that a Repository Checkout task is executed beforehand
- select a repository:
  - if the task is used in a build plan, you can select the default repository of that plan
  - if the task is used in a deployment, you can specify the working directory into which the repository was checked out
  - in either case you can choose a repository using its name
- provide the branch name, you may also use Bamboo variables

---

# Task: Repository Commit

> YAML Specs does not support Repository Commit task.

The **Repository Commit** task enables to commit changes to the checked out repository. The task will commit all local modifications. For DVCS repositories, the task will push the new commit to the remote repository.

To use the task:

- make sure that a Repository Checkout task is executed beforehand
- select a repository:
  - if the task is used in a build plan, you can select the default repository of that plan
  - if the task is used in a deployment, you can specify the working directory into which the repository was checked out
  - in either case you can choose a repository using its name
- provide the commit message, you may also use Bamboo variables

The commit task was designed to help in simple use cases. For DVCS repositories, you will have more control over pushing changes to the remote with the Repository Push task.

---

# Task: Repository Push

> YAML Specs does not support Repository Push task.

The **Repository Push** task enables pushing local commits to a remote repository. Only applicable to DVCS repositories, which distinguish between local and remote commits.

To use the task:

- make sure that a Repository Checkout task is executed beforehand
- select a repository:
  - if the task is used in a build plan, you can select the default repository of that plan
  - if the task is used in a deployment, you can specify the working directory into which the repository was checked out
  - in either case you can choose a repository using its name

The push task was designed to be used when solving more complex problems:

- pushing commits created by other tasks
- pushing custom commits (e.g. cryptographically signed or with a custom author)
- pushing merge results
- transactional processing through a build (multiple commits with a single push at the end)

If a simple “commit and push” operation is sufficient, consider using the Repository Commit task.

---

# Task: Maven

> Simple Maven 3 task running with JDK 1.8.
>
> Only Maven 3.x tasks are supported. Maven 1.x and Maven 2.x tasks should be replaced with Script tasks.

```yaml
---
version: 2
# ...
  tasks:
    - maven:
        executable: Maven 3.2
        jdk: JDK 1.8
        goal: clean install
        tests: false
```

The **Maven** task in Java Specs provides a dedicated task to execute Maven goals for building, testing and deploying your code. Apache Maven is a tool used for building and managing Java-based projects.

The most important configuration option for Maven task is the goal to execute. Multiple goals can be specified, separated by spaces, for example `clean install`. It is also possible to pass additional JVM and Maven parameters, such as `-Djava.awt.headless=true` or `-Pdatabase.tests`. Bamboo Variables may be included in this field too, for example `clean compile ${bamboo.maven.compile.flags}`.

Two other necessary fields to set are the JDK and the Maven executable. These configuration options indicate which version of Java Development Kit and which version of Apache Maven should be used to execute the task. The options should point to valid Bamboo capabilities.

More detailed documentation of the Maven task can be found here.

#### PARSING TEST RESULTS

Maven 3 task with test parsing enabled.

```yaml
---
version: 2
# ...
  tasks:
    - maven:
        executable: Maven 3.2
        jdk: JDK 1.8
        goal: clean install
        tests: '**/my-acceptance-tests/target/surefire-reports/*.xml'
```

The **Maven** task is capable of parsing test results generated during the build. Bamboo will use JUnit test parser to gather test results.

#### CONFIGURING RUNTIME ENVIRONMENT

Maven 3 task running in custom working directory, with `MAVEN_OPTS` environment variable set.

```yaml
---
version: 2
# ...
  tasks:
    - maven:
        executable: Maven 3.2
        jdk: JDK 1.8
        goal: clean install
        tests: false
        environment: MAVEN_OPTS="-Xmx768m -Xms64m -Dmaven.compiler.fork=true"
        working-dir: maven-working-dir
```

The **Maven** task offers additional flexibility for configuring runtime environment.

There is a possibility of setting environment variables for the build. It is a common practice to set `MAVEN_OPTS` for the build's execution (e.g. `MAVEN_OPTS="-Xms200m -Xmx700m"`). You can include Bamboo variables for this configuration option (e.g. `MAVEN_OPTS="${bamboo.maven.compile.opts}"`).

Multiple environment variables can be provided, separated with spaces. Parameters with spaces must be quoted. Note that existing environment variables are automatically available and there is no need to override them.

It is also possible to execute the Maven task in a specific sub-folder of the build's working directory by providing a relative path to it.

# Task: Docker CLI

> YAML Specs does not support Docker CLI tasks.

The **Docker** task in Java Specs provides support for Docker using a dedicated DockerCLI task. The Docker CLI task is a task which allows for building, pulling from registry, pushing to registry Docker images and running Docker images. To improve its readability, Docker CLI is split into four separate Bamboo Specs builders.

#### DOCKERBUILDIMAGETASK

> YAML Specs does not support Docker CLI tasks.

The **DockerBuildImageTask** task is used to build the Docker image. The `Dockerfile` content can be inlined or it can be retrieved from a working directory of a task.

#### DOCKERRUNCONTAINERTASK

> YAML Specs does not support Docker CLI tasks.

The **DockerRunContainerTask** task is used to run images in a container. This task provide powerful customization options such as:

- detaching containers
- linking other detached containers running in the same Bamboo job
- container environment variables, command or working directory
- waiting for a service to start
- port or volume mappings

Note, by default when builder is created, the container working directory is set to `*/data*` and default host-container volume mapping is set to `*${bamboo.working.directory}*` -> `*/data*`

#### DOCKERPUSHIMAGETASK

> YAML Specs does not support Docker CLI tasks.

The **DockerPushImageTask** task is designed to push an image to a Docker registry. You can use it to push your image to both: your own Docker registry and Docker Hub. Please note that, if you are using a custom registry, you need to specify the full repository tag for Docker image:

`registry.address:port/namespace/repository:tag`

Docker push by default uses credentials stored in the agent's `*~/.dockercfg*`, however you may specify explicit username, password and email which will be used to authenticate against Docker registry.

#### DOCKERPULLIMAGETASK

> YAML Specs does not support Docker CLI tasks.

The **DockerPullImageTask** task is similar to DockerPushImageTask, however, it's used for pulling Docker images from the registry.

---

# Task: NUnit

NUnit is a unit-testing framework for all .Net languages.

The **NUnit** task provides support for NUnit with parser and runner tasks.

#### NUNIT PARSER

> NUnit Parser task in a typical setup.

```
---
version: 2
# ...
  final-tasks:
```

```
    - test-parser:
        type: nunit
        test-results: 'test-results/*.xml'
```

.NET builder tasks in Bamboo (e.g. NAnt) don't parse test information as part of the task. You must configure a NUnit Parser task for the test results from the builder task to be parsed.

NUnit Parser task builder works in a very similar way to the JUnit parser task builder.

#### NUNIT RUNNER

The NUnit Runner task runs the tests defined with the NUnit framework. The NUnit version 3 has been completely rewritten since version 2, so you have to specify version of the executable you wish to use.

---

## Conditional tasks

Every task might have a condition. If the condition isn't met, task execution will be skipped and Bamboo will try to run the next task in job.

Task executed if planRepository.branch variable is `development`.

```
version: 2
# ...
tasks:
  - script:
      interpreter: SHELL
      scripts:
        - echo hello
      conditions:
        - variable:
            equals:
              planRepository.branch: development
```

---

## Configuring unsupported tasks

The 'Variable increment' task for `bamboo.variable.name` variable using generic AnyTask.

```
- any-task:
    plugin-key: com.atlassian.bamboo.plugins.variable.updater.variable-updater-generic:variable-updater
    configuration:
      variable: bamboo.variable.name
      strategy: DEPLOYMENT
      variableScope: JOB
```

Some tasks, like third-party tasks from plugins, are not supported natively by Java and YAML Specs.

In order to handle an unsupported task in Bamboo Specs, you should use the generic **AnyTask** class or **any-task** keyword. The **AnyTask** class or **any-task** keyword require an identifier of a task module and a set of properties describing task's settings.

The fastest way to get this data is as follows:

1. **Create a plan in Bamboo**. From top menu choose *'Create'* > *'Create a new plan'*, enter plan name and key, select *'Link new repository'* with *'None'* and click *'Configure plan'*.
2. **Add a task you would like to use**. Click *'Add task'* and select a task from list, set all options, click *'Save'* and next click *'Create'*.

3. **Export plan configuration**. From the plan configuration page click *'Actions'* > *'View plan as Java Specs'* or *'View plan as YAML'*.

4. **Copy generated AnyTask code to your Bamboo Specs**. Hint: you may want to encapsulate it in your own class inheriting from the Task class.

> Avoid using AnyTask for tasks which are already available in Bamboo Specs. Always check Bamboo release notes and upgrade guide to see if support for new tasks has been added to Bamboo. If so, replace your AnyTasks with the official implementation.
> Check release notes of third party plugin vendors as they can also prepare official classes for Bamboo Specs. This will ensure that configuration you use is compatible with Bamboo and up to date.

# Variables

Defining project variables:

Defining plan variables:

```yaml
---
version: 2
# ...
variables:
  username: admin
  releaseType: milestone
```

```yaml
---
version: 2
# ...
variables: # In case of storing raw passwords, import will fail.
  password: BAMSCRT@@@@@r7STxYwtyNCz123WWXKq8g==
```

Using variables in tasks:

```yaml
---
version: 2
Default job:
  tasks:
    - script: echo 'Working branch is $bamboo_planRepository_1_branch'
```

Variables can be used to make values available when building plans in Bamboo.

| | |
|---|---|
| **Build-specific variables** | Evaluated by Bamboo dynamically at build time. For example, `bamboo.buildNumber`. |
| **System variables** | Apply across your entire Bamboo instance and inherit their values from system or environment variables of the same name. For example, `system.M2_HOME`. |
| **Global variables** | Defined across your entire Bamboo instance, and have the same static value for every plan that is built by Bamboo. |
| **Project variables** | Defined for a project and have the same static value for every plan within this project (unless it has been overriden on a plan level). |

| | |
|---|---|
| **Plan variables** | Similar to global variables, but are defined for specific plans. Plan variables override global variables with the same name. |

Variables can be used in configuration fields of tasks. The following format should be used when referencing a variable:

| | |
|---|---|
| `${bamboo.variableName}` | Build-specific, global, project, and plan variables. |
| `${system.variableName}` | System variables. |

For the Script task, Bamboo variables are additionally exported as shell variables. All full stops (periods) are converted to underscores. For example, the variable `bamboo.my.variable` will be transformed to `$bamboo_my_variable`.

To learn more about Bamboo plan variables, see Bamboo variables documentation.

In case of storing passwords in variables, you can use Bamboo Specs Encryption and store it in repository in an encrypted form. See documentation of sensitive data encryption.

# Labels

Defining plan labels:

```
---
version: 2
# ...
labels:
  - release
  - team-bamboo
```

Labels provide a convenient way to tag and group plans that are logically related to each other. Labeling allows you to filter the plans displayed on the Bamboo Dashboard or Wallboard. You may want to do this if you have set up a large number of plans in your Bamboo instance and want to highlight specific ones for attention.

For example, you may want to label all builds related to the release with a `release` label. You can then filter your wallboard during your release, to display only the relevant builds.

## Removing labels

Removing plan labels:

```
---
version: 2
# ...
labels: []
```

In older Bamboo versions, labels could have been added to Specs-managed plans by other means, for example via REST calls.

You can explicitly instrument Bamboo to remove all labels from a plan. If you neither clear nor define labels in your Bamboo Specs, Bamboo will not change the current plan's labels.

# Docker

By default, Docker is off for jobs and environments in YAML.

To disable Docker in YAML, remove the configuration from the file.

Enabled Docker configuration with minimum settings.

```
---
version: 2
# ...
docker: oracle
```

How to set Docker configuration to a job or to an environment.

```
---
version: 2

deployment:
  name: Deliver to planet
  source-plan: PLAN-MAR

release-naming:
  next-version-name: 0.${bamboo.buildNumber}
environments:
  - QA

QA:
  tasks:
    - script:
      - echo 'Release planet'
  docker:
    image: ubuntu
```

Configuration from plan level for every job in the plan.

```
---
version: 2

plan:
  project-key: ROCK
  key: COMET
  name: Comet mission

docker: ubuntu
```

Override Docker settings from plan in job.

```
---
version: 2

plan:
  project-key: ROCKET
  key: LAUNCH
  name: Launch Rocket

docker: ubuntu

# ...

Build Ship:
  tasks:
```

```
    - script:
      - echo 'Building ship...'
      - echo 'Completed!'
  docker:
    image: oracle
```

Builds and deployments are normally run on the Bamboo agent's native operating system. However, both jobs and environments can be instrumented to be run in a **Docker container**, for better control over the available tool set and additional build isolation. Docker is a tool which automates the running and deployment of applications inside containers.

The Docker image can be configured on a plan level and on the job level. The configuration from the job level overrides the Docker settings from plan level.

---

# Image

A simple enabled Docker configuration with specified Docker image.

```
---
version: 2
# ...
docker: oracle
```

```
---
version: 2
docker:
  image: oracle
```

To run a job or an environment in a Docker container, you will need to specify which Docker image to use.

You can choose the repository host, namespace and tag for the image, by following the standard Docker image format:

`localhost:5000/namespace/image:tag`

Bamboo variables are allowed in the Docker image configuration field.

---

# Volumes

Default Docker volumes.

```
---
version: 2
# ...
  mapping:
    - ${bamboo.working.directory}: ${bamboo.working.directory}
    - ${bamboo.tmp.directory}: ${bamboo.tmp.directory}
```

Docker configuration without the default data volumes.

```
---
version: 2

docker:
  image: ubuntu
  volumes:
    "/bin": "~/bin"
```

```
    "/etc": "~/etc"
  use-default-volumes: false
```

You may choose which agent's directories should be mounted in the Docker container under specific paths. This way you can define additional data volumes for the Docker container.

By default, Bamboo maps some of the agent's directories to corresponding directories in the Docker container. You can disable this behaviour if it's undesirable.

Bamboo variables are also allowed in data volumes configuration fields.

## Run arguments

You can provide additional arguments for the 'docker run' command, for example 'host' or 'privilege'. For the complete list of arguments available for this command, see the Docker documentation.

```
---
version: 2



docker:
  image: ubuntu
  docker-run-arguments:
    - --net=host
```

# Triggers

Triggers in Bamboo allow plan builds to be started automatically.

For Specs YAML, by default, Bamboo polls the repository every 3 minutes. If your plan repository is Bitbucket Server, the Bitbucket Server trigger will be used instead.

## Polling the repository for changes

```
---
version: 2
# Short syntax
triggers:
  - polling: 150
```

```
---
version: 2
# Long syntax
triggers:
  - polling:
      period: 150
```

Bamboo will poll the selected source code repositories for code changes, using either a specified interval or a schedule. If Bamboo detects code changes, a build of the plan is triggered.

#### POLL PERIODICALLY

You can poll source code repository every **N** minutes / hours / days for new commits.

#### POLL DAILY, WEEKLY OR MONTHLY

You can define a time schedule when to poll the repository. You can poll daily, on specific days of a week or on specific days of a month.

#### POLL WITH CRON EXPRESSION

```
---
version: 2
triggers:
  - polling:
      cron: 0 0/30 9-19 ? * MON-FRI
```

You can also use a cron expression (see Constructing a cron expression in Bamboo).

---

## After successful deployment trigger the plan

Bamboo will trigger the plan after a successful deployment of the chosen environment or any environment in the deployment project.

#### AFTER DEPLOYMENT OF THE CERTAIN ENVIRONMENT

```
---
version: 2
triggers:
  - after-deployment:
      deployment-project: release
      environment: production
```

#### AFTER DEPLOYMENT OF ANY ENVIRONMENT

```
---
version: 2
triggers:
  - after-deployment:
      deployment-project: release
```

---

## Repository triggers a build on commit

A plan listening for events from a `my-repo` repository from `12.34.56.78` IP address.

```
---
version: 2
triggers:
  - remote: 12.34.56.78
```

```
---
version: 2
triggers:
```

```
  - remote:
      ip: 12.34.56.78
```

Listening to `localhost`.

```
---
version: 2
triggers:
  # Leaving `remote` empty, Bamboo will allow only localhost to execute a trigger
  - remote
```

Bamboo waits to receive a message about changed code from any of the selected source code repositories. When Bamboo receives such a message, a build of the plan is triggered. This option minimizes server load, because message events are sent only when code changes to a repository are committed. This is the default option when you use a linked Bitbucket Server repository.

See the Repository triggers the build when changes are committed how to set up the trigger on the repository side.

On the Bamboo side, use the `triggers(new RemoteTrigger())` on plan definition. By default the remote trigger will react on all triggering repositories defined in a plan. You can limit it by selecting a subset of repositories.

In case you want Bamboo to trigger on post-commit messages from other than the primary IP address for the repository, specify it in `triggerIPAddresses`.

Notes:

- the repositories listed in remote trigger must be among those declared in a plan
- if you use a Git repository then you must type the IP address of your repository host
- for Bitbucket Cloud the current outbound IP addresses can be found at Access Bitbucket Cloud from Behind a Firewall
- if you're using the Bitbucket Cloud Bamboo post-push hook, ensure that the user you are using to authenticate triggering the build has the 'build' permission on the plan you are attempting to trigger. Also see Triggering a Bamboo build from Bitbucket Cloud using Webhooks

---

# Cron-based scheduling

In case of the cron-based scheduling, the build will always run at specified time, no matter whether content of repositories has changed or not.

#### SCHEDULE WITH TIME INTERVAL

YAML Specs does not support scheduling with time intervals.

You can schedule a build to run every N minutes / hours / days.

#### SCHEDULE WITH DAILY, WEEKLY OR MONTHLY PLAN

YAML Specs does not support this trigger.

#### SCHEDULE WITH CRON EXPRESSION

```
---
version: 2

#short syntax
triggers:
  - cron: 0 0 0/12 ? * *
```

```
---
version: 2

#long syntax
triggers:
```

```
  - cron:
      expression: 0 0 0/12 ? * *
```

The schedule can be also expressed using a cron expression.

---

# Conditional build triggers

In Bamboo, you can define trigger conditions, which for example will make a given trigger execute only if specified plans are passing.

```
version: 2
#...
triggers:
  - polling:
      period: 180
      conditions:
      - all-other-conditions:
          custom.rejectBranchBuildWithoutChange.enabled: true
```

#### RUNNING A BUILD WHEN ANOTHER PLAN HAS SUCCESSFULLY COMPLETED

This trigger condition allows to execute plan only if another plan has successful last build result.

> Trigger plan only when changes at repository detected and “PROJECTKEY-PLANKEY” latest result is green.

```
version: 2
#...
triggers:
- polling:
    period: 180
    conditions:
    - green-plan:
      - PROJECTKEY-PLANKEY
```

---

# Multiple triggers

```
---
version: 2
triggers:
  - polling: 130
  - cron: 0 * * * ? *
  - remote: 192.168.0.1
```

You can use many triggers for a single plan. In such case, a build will run for a plan whenever condition for any of the triggers is satisfied.

---

# Triggering selected repositories

```
version: 2
#...
repositories:
  - Local git:
```

```
      type: git
      url: http://localhost:7990/local.git
  - Bitbucket demo:
      type: bitbucket
      slug: atlassian/demo
      branch: master
triggers:
  - polling:
      period: 3600
      repositories:
        - Local git
```

It's possible to configure triggers to react on selected repositories if plan has several of them.

---

## No triggers

```
---
version: 2
triggers: []
```

You can also configure your plan to have no triggers at all. In such case, the plan must be executed manually.

This is useful when human decision is required, for instance publishing a new product release.

---

# Plan branches

---

Plan branches are used to represent a branch in your version control repository, with the plan branch using the same build configuration as your plan. Tools such as Git encourage a practice called feature branching, where a developer can use a new branch to work in isolation from his or her team members before merging their changes back into main line development.

With plan branches in Bamboo:

- any new branch created in the repository can be automatically built and tested using the same build configuration as that of the parent plan
- any branches deleted from the repository can be deleted automatically from Bamboo according to the settings
- you have the flexibility to individually configure branch plans, by overriding the parent plan, if required
- optionally, changes from the feature branch can be automatically merged back to the `master` when the build succeeds

You can create plan branches manually or automatically. The branch configuration can be provided on the plan level and customized on the branch level. The settings provided in the branch configuration override the settings provided for the plan.

---

## Automatic branch management

Plan branches can be created and deleted automatically based on the updates in the primary source repository. Automatic branch management is available for Git and Subversion. For other repository types, you can use manual branching. You can override the default settings for a branch, such as values of the variables.

By default, automatic branch management is:

- disabled for branches that you create manually
- enabled for branches that are created automatically

You can specify how often Bamboo checks the primary source repository for new or deleted branches. The default value is 300 seconds. You can also override the branch deletion settings.

#### HANDLING NEW BRANCHES

Create new plan branch for every repository branch. It's the default option.

```yaml
---
version: 2
branches:
  create: for-new-branch
```

Create new plans for new branches matching Jira issue key.

```yaml
---
version: 2
branches:
  create:
    for-new-branch: JIRA-[0-9]+
```

Create new branch for new pull request

```yaml
---
version: 2
branches:
  create: for-pull-request
```

Create new branch for new pull request and allow pull requests from forked repositories

```yaml
---
version: 2
branches:
  create:
    for-pull-request:
      accept-fork: true
```

These are the options to handle new branch creation:

| | |
|---|---|
| **Don't create new branches (Java Specs default)** | Bamboo will not create new plan branches automatically when new branches are detected in the primary source repository. |
| **Create for all new branches (YAML Specs default)** | Create a plan branch for each new branch detected in the primary source repository use the `planBranchCreation` method with `new PlanBranchCreation()`. |
| **Create for selected branches** | Create a plan branch for each new branch detected in the primary source repository that matches the regular expression that you provided use `planBranchCreation()` with the `new PlanBranchCreation().matchingPattern()`. |

#### HANDLING DELETED BRANCHES

Create plan branches for all new branches in the repository. Delete plan 30 days after a repository branch is deleted.

```yaml
---
version: 2
branches:
  delete:
    after-deleted-days: 30 # default value is 1
```

Never remove plan branch for deleted repository branches.

```yaml
---
version: 2
branches:
  delete:
    after-deleted-days: never
```

Turn off plan branches removal. It's default for Java specs.

```yaml
---
version: 2
branches:
  delete: never
```

These are the options to handle branch deletion:

| | |
|---|---|
| **Don't delete (Java Specs default)** | If a branch in the primary source repository was deleted, Bamboo does not automatically delete the corresponding plan branch. |
| **With a daily cleanup** | If a branch in the primary source repository was deleted, the corresponding plan branch is removed with a daily cleanup that happens daily at 3 AM (server time) - use `removedBranchCleanup` with `new BranchCleanup()`. |
| **After period of time(default 1 day for YAML Specs)** | If a branch in the primary source repository was deleted, Bamboo deletes the corresponding plan branch after N days - use `removedBranchCleanup` with `new BranchCleanup().periodInDays(N)`. |

#### HANDLING INACTIVE BRANCHES

Create plan branches for all new branches in the repository. Delete plans after 60 days of no activity on a repository branch.

```yaml
---
version: 2
branches:
  delete:
    after-inactive-days: 60 # default value is 30
```

Never remove plan branch for repository branches without activity.

```yaml
---
version: 2
branches:
  delete:
    after-inactive-days: never
```

These are the options to handle inactive branches:

| | |
|---|---|
| **Don't delete (Java Specs default)** | If a branch in the primary source repository is inactive, Bamboo does not automatically delete the corresponding plan branch. |
| **Delete after period of inactivity(default 30 days for YAML Specs)** | If a branch in the primary source repository is inactive, Bamboo deletes the corresponding plan branch if no commits are detected for the specified period. The value must be higher than 0 and is specified in days. |

# Manual branch management

```
---
version: 2
branches:
  create: manually
```

Bamboo allows to manually create plan branches for already existing branches in a repository. Bamboo Specs does not have an option to manually specify plan branches at the moment. You need to use Bamboo to manually create branches after plan is created.

Alternatively, you could create separate plans for each branch. Keep in mind that using this approach you will not be able to use Bamboo branch-specific features (for instance to see them on the “Branch view” page).

# Automatic branch merging

Bamboo provides two merging models if you choose to automate your branch merging. Automatic merging has few limitations:

- it can only be used with Git repositories
- it can only be used with branches that were configured in Bamboo
- it cannot be used with the Git implementation embedded in Bamboo - you have to set up native Git

**BRANCH UPDATER**

Create a plan building the ‘master’ branch.

Create plan branches automatically for newly created feature branches.

Merge the master branch into a feature branch and build the merge result. Push a merge commit into the feature branch on a successful build.

```
---
version: 2

branches:
  create:
    for-new-branch: feature/.*
  integration:
    # plan branch name. At RSS world it will be repo branch name.
    # Bamboo tries to find by plan branch name, if not found then by VCS branch name,
    # if not found then merge with "master" plan.
    merge-from: master
    push-on-success: true
```

A branch repository is kept up-to-date with changes to master. The Branch updater should be used when you want to:

- automatically merge changes from the team’s master branch into your feature branch, after a successful build of the master branch
- get notified when the changes on your feature branch are no longer compatible with the team’s master branch

**GATEKEEPER**

Create a plan building the ‘master’ branch.

Create plan branches automatically for the newly created `integration` branch.

Merge the integration branch into ‘master’ branch and build the merge result. Push a merge commit into the master on a successful build.

```
---
version: 2
```

```
branches:
  create:
    for-new-branch: feature/.*
  integration:
    # plan branch name. At RSS world it will be repo branch name.
    # Bamboo tries to find by plan branch name, if not found then by VCS branch name,
    # if not found then merge with "master" plan.
    merge-to: master
    push-on-success: true
```

In Gatekeeper the default repository is only updated with changes in the branch that have built successfully. The Gatekeeper should be used when you want to:

- automatically merge your feature branch back into the team's master branch, after a successful build of the merged changes from both branches
- get notified when a build of combined changes from both branches fails, preventing the feature branch from being merged back into the team's master branch

---

# Branch notifications

YAML doesn't support branch notifications.

You can get build notifications from branch plans just as you do for master plans. You can choose one of the following options:

- notify committers and people who have favorited this branch
- use the plan's notification settings
- don't send notifications for branches

---

# Plan branch specific settings

When using RSS, it is possible to have a different build configuration for each plan branch. These plan branches are called "Specs branches".

Specs branches are configured according to the content of the Bamboo Specs stored on that branch. This means that the default values defined in the Plan Branch Management section of Bamboo Specs, such as triggers, notifications and branch integration are ignored and respective Bamboo Specs are applied instead.

If a specific Bamboo Specs plan is not defined (or published) on a branch, the plan branch inherits its configuration from the default branch.

You can read enhanced plan branches for more details.

---

# Overriding branch settings using Bamboo Specs

Defining new stage and job for `integration-branch`.

```
---
version: 2
plan:
  project-key: ROCKET
  name: Launch Rocket
  key: LAUNCH

stages:
  - Default stage:
```

```
      - Default job

Default job:
  tasks:
    - script: echo 'Default job'
    - clean

branch-overrides:
  -
    integration-branch:
      stages:
        - Different stage:
            - Different job
      Different job:
        tasks:
          - script: echo 'Override job'
```

Using regular expressions for branch matching.

```
---
version: 2
# ...

branch-overrides:
  -
    bugfix/.*: ...
  -
    # More specifics will be matched first
    feature/BAM-1234-work-in-progress: ...
  -
    feature/.*: ...
```

You can define these keys on the `branch-overrides` key.

```
---
version: 2
# ...

branch-overrides:
  -
    hotfix/.*:
      # All of these keys are override-able and will be merged
      # if the current branch is "integration-branch"
      docker: ...
      triggers: ...
      notifications: ...
      variables: ...
      labels: ...
      branch-config: ...
  -
    issues/.*:
      # Stages are also override-able
      stages:
        -
          Build Stage:
            jobs:
              - Ship Server
              - Another Ship Server
      # And jobs
      Ship Server: ...
      Another Ship Server: ...
```

In some scenarios, having different configuration on plan branches is not enough. Managing configuration for long-standing branches, managing merge conflicts, or having default configuration on fresh branches is still difficult if you are not using Java Specs. Since Bamboo 7.1.0, the key `branch-overrides` is available and allows more reusability between plan branches.

The branch override key defines configuration specifically for the branch to which Bamboo Specs are applied. It matches the branch name with the regular expressions defined and decides which configuration will get applied.

The override is decided by the first matching regex from the list of matches, from top to bottom. The more specific regular expressions matching branches should precede the more generic, be at the top.

Specifically, this key overrides the matched branch rules exclusively on the branch which is currently being updated by Bamboo Specs. The branch override does not have side-effects on other branches or change how your Specs are processed at runtime.

The override does a shallow merge of the root keys in the YAML document and ignores the previous overriden content. **The new configuration must be complete**. To avoid repetition, you can use YAML anchors.

---

# Conditional execution of specs on branches

Refer to the previous section branch-overrides.

It is possible to alter behavior of Bamboo Specs depending on the branch from which they are run. This can be used to define default configuration for branches that differs from configuration of 'master' plan and is applied to all branches as soon as they are created. Another application is preventing branch specific configuration from being accidentally applied to other branches after a merge.

---

# Branch specific configuration

When using specs on branches, it is possible disable automatic branch expiry and configure automatic branch management for the current branch. The format of automatic branch configuration is identical to the one used in Plan Branch Management section of the `master` plan configuration.

Configure branch integration.

```
version: 2

plan:
  project-key: PRJ
  key: PLAN
  name: Plan name

stages:
  - Default stage:
      - Default job

Default job:
  tasks: []

triggers: []

branch-config:
  integration:
    merge-from: release
    push-on-success: false
  disable-expiry: true```
```

---

# Overriding branch settings in the UI

If branch configuration is not managed by specs then, whether a plan branch is created automatically or manually, the `master` branch maintains the structure and configuration of its branch plans. You can use Bamboo to override settings in a branch plan, see Branch

details configuration for more details.

# Requirements

Configuring a job with a simple Script task usually requires specifying the requirements manually to ensure necessary capabilities exist on the agent.

```
---
version: 2
# ...
Build Rocket:
  tasks:
    - script: ${system.builder.mvn3.Maven 3} clean test
  requirements:
    - Maven 3
```

```
---
version: 2
# ...
Build satellite:
  tasks:
    - script: ${system.builder.command.Python 3} build
  requirements:
    - system.builder.command.Python 3
```

Configure jobs with different types of requirements

```
---
version: 2
# ...
Build satellite:
  tasks:
    - script: echo hello
  requirements:
    - hasDocker # requirement 'hasDocker' exists
    - os: Ubuntu \d+ # requirement 'os' with value matches pattern exists, e.g. Ubuntu 18
```

A **Requirement** is specified in a job or a task. A requirement specifies a capability that an agent must have for it to build that job or task. A job inherits all of the requirements specified in its tasks.

Defining proper requirements for a build agent is necessary to run the same build on different platforms, such as operating systems or databases, or in the same platform with with different versions to check the compatibility matrix.

There are four types of capabilities in Bamboo that can be specified by job and task requirements:

| | |
|---|---|
| **Executable capabilities** | Define external programs that can be called by Bamboo, for example Ant, Maven, MSBuild or PHPUnit. |
| **JDK capabilities** | Define the JDK versions to be used by the job or task. |
| **Version control capabilities** | Specify the VCS client application that Bamboo should use to checkout source code. |
| **Custom capabilities** | Used to control which jobs will be built by a particular agent. |

You may use **Custom capabilities** if the builds for a particular job should only run in a Windows environment; You could create a custom capability of `operating.system = Windows` for the appropriate agent(s), and specify it as a requirement for this job.

To learn more, refer to the following documentation pages:

- Agents and capabilities
- Configuring a job's requirements

Additionally, in Bamboo variables documentation you may find details on how to refer to various types of capabilities.

# Repositories

This section describes how to configure Bamboo to use repositories. You can specify repositories at the following levels in Bamboo:

| | |
|---|---|
| **Linked** | Repositories are available to all plans in Bamboo. |
| **Project** | Repositories are available to all plans in project. |
| **Plan** | Repositories are available in one Bamboo plan. |

For YAML Specs, if repositories are not defined explicitly, Bamboo automatically adds the repository to the created plan. Additionally, Bamboos adds the first task as a checkout task from this repository if there's no checkout task defined for a job.

## Multiple repositories

A plan with two linked repositories.

```yaml
---
version: 2
# ...
repositories:
  - my-global-repository1
  - my-global-repository2
```

A plan without repositories. Checkout task is not added.

```yaml
---
version: 2
# ...
repositories: []
```

One or more repositories can be added to a plan. All those repositories will be available to every job in the plan. The first repository in the list is the Plan's Default Repository.

## Plan-local vs linked vs project repositories

Linked repositories are available globally to all plans and jobs configured on the Bamboo server. Doing this can save you from having to reconfigure the source repositories in multiple places if these ever change - any changes to a linked repository are applied to every plan or job that uses that repository.

> This feature is available for Bamboo DC licenses only.

Project repositories are available for all plans and jobs at same project where repository defined.

#### CREATING LINKED REPOSITORIES

While it's currently not possible to create new linked repositories using RSS, you can do it manually. See Create linked repository with Bamboo Java Specs.

See Linking to source code repositories on how to set up linked repositories using Bamboo. Once linked repository is defined, you can refer it in your code, see chapter below.

#### REFERRING LINKED REPOSITORIES

```
version: 2
#...
repositories:
- my-global-repository
- my-second-global-repository:
    scope: global
```

Once the linked repository is set up you can easily refer to it in your plan configuration. When using specs on branches the same construct can be used to refer to plan-defined repositories, which prevents unnecessary duplication of the repository configuration.

#### CREATING PROJECT REPOSITORIES

> This feature is available for Bamboo DC licenses only.

You can create a project repository available for all the plans in this project.

#### REFERRING PROJECT REPOSITORIES

```
version: 2
#...
repositories:
- my-project-repository
- my-second-project-repository:
    scope: project
```

> This feature is available for Bamboo DC licenses only.

Once the project repository is set up you can easily refer to it in your plan configuration.

It's possible to reference linked or project repository by name only. First Bamboo tries to find repository at project, if not found it will look at linked repositories. If repository with same name is present at project and global, then error will be thrown and scope (*project* or *global*) should be provided.

#### OVERRIDING BRANCH OF A LINKED REPOSITORY

When referring to linked (or, in case of branches, plan) repository it is possible to override the branch. Note however, that when using specs on branches, the override of configuration of the default repository is ignored, as it must match the current specs branch.

```
version: 2
```

```yaml
plan:
  project-key: PRJ
  key: PLAN
  name: Plan name
  master-branch:
    name: branchX
    display-name: branchX

stages:
  - Default stage:
      - Default job

Default job:
  tasks: []

triggers: []
```

#### CREATING PLAN REPOSITORIES

```yaml
version: 2
#...
repositories:
  - my-plan-git-repository:
      type: git
      url: ssh://git@bitbucket.org:my-company/my-repository.git
      branch: master
  - my-plan-bitbucket-cloud-repository:
      type: bitbucket
      slug: atlassian/bamboo-specs
      branch: master
      viewer: com.atlassian.bamboo.plugins.atlassian-bamboo-plugin-bitbucket:bbCloudViewer
```

Creates a local repository for the plan. This repository will not be shared among other plans.

---

# Git

You need to have previously defined a Git capability before you can configure a Git source repository – see defining a new version control capability.

You need to provide a Git executable to use symbolic links, submodules, automatic branch detection and automatic merging. These are not supported by the Bamboo built-in Git.

#### GIT: BASIC OPTIONS

In case you are using a linked repository, you need to provide only the repository name.

In case you are creating a local repository definition, you have to provide at least:

- **name** to help identify repository in Bamboo
- **URL** to your Git repository; valid forms are:
  - `git://host.xz[:port]/path/to/repo.git`
  - `ssh://[user@]host.xz[:port]/path/to/repo.git`
  - `[user@]host.xz[:port]/path/to/repo.git`
  - `http[s]://host.xz[:port]/path/to/repo.git`
  - `/path/to/repo.git`
  - `file:///path/to/repo.git`
- **Branch** - the name of the branch (or tag) you want to work on

#### GIT: AUTHENTICATION

In case you have to authenticate you can use several methods.

#### WITH SSH AUTHENTICATION (RECOMMENDED)

Git repository with SSH authentication.

```yaml
version: 2
#...
repositories:
- my-repository:
  type: git
  url: ssh://git@bitbucket.org:my-company/my-repository.git
  branch: master
  ssh-key: ENCRYPTED_KEY
  ssh-key-passphrase: ENCRYPTED_PASSPHRASE
```

Provide an SSH private key and provide the SSH passphrase.

#### WITH USER/PASSWORD AUTHENTICATION

Git repository with user-password authentication.

```yaml
version: 2
#...
repositories:
- my-repository:
  type: git
  url: https://bitbucket.org/my-company/my-repository.git
  branch: master
  username: username
  password: ENCRYPTED_PASSWORD
```

We advise to not store user and password in the source code, but to read them from a resource file or a system property. You can also use the Bamboo Specs Encryption to store sensitive data.

#### WITH GLOBAL SHARED CREDENTIALS

Git repository with authentication using global shared credentials.

```yaml
version: 2
#...
repositories:
- my-repository:
    type: git
    url: ssh://git@bitbucket.org:my-company/my-repository.git
    branch: master
    shared-credentials: identifier
```

You can store global credentials within Bamboo for easier access to repositories. The access details that you provide are available to all plans on your Bamboo server. By default scope for shared credentials is set to global. See Shared credentials for more details.

#### WITH PROJECT SHARED CREDENTIALS

Git repository with authentication using project shared credentials.

```yaml
version: 2
#...
repositories:
- my-repository:
    type: git
    url: ssh://git@bitbucket.org:my-company/my-repository.git
    branch: master
    shared-credentials:
      name: identifier
      scope: project
```

This feature is available for Bamboo DC licenses only.

You can store project shared credentials to access your repositories. The access details that you provide are only available to all plans within your project.

#### WITH NO CREDENTIALS

In case you want to access the repository anonymously, simply don't declare any authentication.

#### GIT: ADVANCED OPTIONS

Checkout shallow clone of Git LFS-enabled repository of an 'integration' branch. The repository doesn't use Git submodules.

```yaml
version: 2
# ...
repositories:
- repository-name-in-bamboo:
  type: git
  url: ssh://git@bitbucket.org:my-company/my-repository.git
  branch: integration
  lfs: true
  use-shallow-clones: true
  submodules: false
```

You can tune repository checkout performance with the following options:

- **shallow clones** - perform shallow clones (i.e. history truncated to a specified number of revisions), this should increase the speed of the initial code checkouts, however if your build depends on the full repository history, we recommend that you don't use this option. Shallow clones are enabled by default.
- **fetch whole repository** - when disabled, Bamboo checks out the current branch (i.e. all commits of the branch); when enabled, Bamboo checks out entire repository (i.e. all commits of all branches)
- **remote agent cache** - when enabled, remote agents will keep repository clones in their caches to speed-up checkout process

Note that not all combinations of the three options above make sense. Reasonable combinations are:

| Shallow clones | Fetch whole repository | Remote agent cache | Outcome |
|---|---|---|---|
| true | false | false | The latest revision of given branch is fetched. |
| false | false | false | All revisions of given branch are fetched. |
| false | false | true | All revisions of given branch are fetched, an agent caches it. |
| false | true | false | The entire repository is fetched. |
| false | true | true | The entire repository is fetched, an agent caches it. |

YAML Specs doesn't support repository viewer configuration.

Other options are:

- **LFS** - enables support for Git Large File Storage (LFS), which replaces large files such as audio samples, videos, datasets, and graphics with text pointers inside Git, while storing the file contents on a remote server. To use this option you must have the following:
  - Git version 1.8.2 or later installed locally in your environment
  - Git LFS 1.2 or later installed.

- **submodules** - select to enable submodules support if these are defined for the repository. If native Git capability is not defined for agent submodules support will be disabled.
- **repository viewer** - if your repository can be viewed in a web browser, select the repository type. This allows links to relevant files to be displayed in the 'Code Changes' section of a build result. For more details see:
  - Integrating Bamboo with Bitbucket Server
  - Integrating Bamboo with Fisheye
- **verbose logs** - outputs more verbose logs from git commands
- **command timeout** - how many minutes are given for git commands to finish (default is 180 minutes).

Group commits from 1 minute, trigger build on `*.java` files only, skip changesets with 'draft' in a commit message.

```yaml
version: 2
# ...
repositories:
- repository-name-in-bamboo:
    type: git
    url: ssh://git@bitbucket.org:my-company/my-repository.git
    branch: integration
    change-detection:
      quiet-period:
        quiet-period-seconds: 60
        max-retries: 5
      exclude-changeset-pattern: .*draft.*
      file-filter-type: include_only
      file-filter-pattern: .*\.java
```

Change detection options:

- **quiet period** - quiet period allows you to delay building after a single commit is detected, aggregating multiple commits per build
- **include/exclude files** - what files Bamboo uses or ignores to detect changes
- **exclude changesets** - a regular expression to match the commit messages to be excluded

---

# Bitbucket Server Git repository

Atlassian Bitbucket Server hosts Git repositories, so you can define it as a standard Git repository. However, defining such repository as Bitbucket Server gives you many advantages.

When you link a repository hosted in Atlassian's Bitbucket Server with a build plan in Bamboo, then without any further configuration:

- Bamboo will automatically run a build when changes are pushed to the Bitbucket Server repository, without needing to configure polling
- Bamboo will automatically update plan branches when a developer pushes a new branch to the repository (or deletes a branch)
- you can click through to Bitbucket Server to see the commit diff for all files that are part of the changeset
- Bitbucket Server commits that are part of a build are displayed in Bamboo
- build results are notified to Bitbucket Server (and displayed for the associated commits and pull requests)

Bitbucket Server and Bamboo only need to have been connected by creating an application link. Repositories in Bitbucket Server are then made available in Bamboo, so it is easy for you to link your Bamboo plan to a Bitbucket Server repository.

When you create a plan that uses a Bitbucket Server source repository, Bamboo will automatically use the repository trigger instead of a polling trigger. This reduces the load on the Bamboo and Bitbucket servers because Bamboo doesn't need to send poll requests (for each branch of each plan) to the Bitbucket Server every few minutes. Instead, Bitbucket Server will notify Bamboo whenever there is a push to the repository.

#### CREATING BITBUCKET SERVER REPOSITORIES IN RSS MODE

In the RSS mode, you must define SSH keys and the clone URL of the repository.

```
version: 2
#...
repositories:
- bbs:
    type: bitbucket-server
    server: bitbucketServerApplink
    project: BBSPROJECT
    slug: my-repository-slug
    clone-url: ssh://my.bitbucket.server:7999/BSSPROJECT/my-bitbucket-repository.git
    public-key: PUBLIC KEY
    private-key: ENCRYPTED_PRIVATE_KEY
    branch: master
    viewer: com.atlassian.bamboo.plugins.stash.atlassian-bamboo-plugin-stash:bbServerViewer```
```

When using Bamboo Specs in interactive mode, the Bamboo server can and fetch the data such as clone url or SSH keys from the Bitbucket Server. In RSS mode, it is not possible, as user session is not available during the automatic update. Therefore the following properties are **mandatory** when in RSS mode:

- **SSH private key** of the repository (the key can be defined in both Bamboo Specs Encryption and plain format).
- **SSH public key** of the repository.
- **SSH clone url** of this repository, as used by git commands (can be found on repository's page in Bitbucket Server).

#### BITBUCKET SERVER: BASIC OPTIONS

In order to configure Bitbucket Server repository provide at least:

| | |
|---|---|
| **Name** | Identifies this repository within Bamboo. |
| **Server** | An application link to Bitbucket Server. |
| **Project** | Project name in Bitbucket Server. |
| **Repository slug** | A repository slug is a URL-friendly version of repository name, automatically generated by Bitbucket for use in the URL. |

For example if your repository name was `føøbar`, in the URL it would become `foobar`. Similarly, `foo bar` would become `foo-bar`. A repository slug is a part of the clone URL you can find in Bitbucket.

#### BITBUCKET SERVER: AUTHENTICATION

Bamboo uses SSH authentication against Bitbucket Server. You can configure:

- **ssh private key** of the repository (the key can be defined in both Bamboo Specs Encryption and plain format)
- **ssh public key** of the repository

You have to provide both private and public key when setting them. If not provided, Bamboo will attempt to generate and install new SSH keys into Bamboo Server instance.

#### BITBUCKET SERVER: ADVANCED OPTIONS

Use shallow clones and enable LFS support.

```
version: 2
#...
repositories:
- bbs:
    type: bitbucket-server
    server: bitbucketServerApplink
    project: BBSPROJECT
    slug: my-repository-slug
    clone-url: ssh://my.bitbucket.server:7999/BSSPROJECT/my-bitbucket-repository.git
    public-key: PUBLIC KEY
    private-key: ENCRYPTED_PRIVATE_KEY
    branch: master
    lfs: true
    use-shallow-clones: true
```

```
    submodules: false
    command-timeout-minutes: 30
    fetch-all: false
    viewer: com.atlassian.bamboo.plugins.stash.atlassian-bamboo-plugin-stash:bbServerViewer
```

Bitbucket Server repository supports the same set of options as Git repository. See the ‘Advanced options’ section for Git for more details.

## SVN

```
version: 2
#...
repositories:
- my-svn-repository:
    plugin-key: com.atlassian.bamboo.plugin.system.repository:svnv2
    server-config:
      repository.svn.authType: password
      repository.svn.branch.create.autodetectPath: true
      repository.svn.repositoryRoot: file:///path/to/my/svn/repository
      repository.svn.useExternals: false
      repository.svn.useExport: false
      repository.svn.userPassword: ENCRYPTED_PASSWORD
      repository.svn.tag.create.autodetectPath: true
    branch-config:
      repository.svn.branch.displayName: svn
```

There are no dedicated options for SVN repositories in Java Specs. The alternative method is to use the generic AnyVcsRepository.

# Artifacts

Artifacts are files created by a job build (e.g. JAR files). Artifact definitions are used to specify which artifacts to keep from a build and are configured for individual jobs. You can also configure artifact sharing between jobs in a plan.

For example, you may want to run acceptance tests on a build, and then share the WAR from one job to another, without rebuilding the WAR each time.

## Defining an artifact

A ‘Test Reports’ artifact containing XML files.

```
---
version: 2
# ...
  artifacts:
    # Define multiple artifacts with the object divisor in YAML
    -
      name: Test Reports
      location: target/reports
      pattern: '**/*.xml'
      required: false
```

You can specify which artifacts to keep by setting up an artifact definition for the job. The artifacts will be available after each build of a job. You can specify:

| | |
|---|---|
| **Name** | Name of the artifact; in case artifact is shared, name must be unique within the plan. |
| **Location** | The relative path to find your artifact; it's a path relative to the workspace directory; don't use absolute paths. |
| **Copy pattern** | Name or Ant pattern of file(s) to keep. |

See also Pattern matching reference.

---

# Sharing an artifact

Declare a shared artifact.

```yaml
---
version: 2
# ...
  artifacts:
    -
      name: Test Reports
      location: target/reports
      pattern: '*.xml'
      required: false
      shared: false
    -
      name: Special Reports
      location: target/reports
      pattern: 'special/*.xml'
      shared: true
```

You can share an artifact among other jobs or plans. Set the `shared` property to `true` and optionally define jobs subscribing to this artifact.

The artifact from the most recent successful build will be used. If there are no successful builds from the artifact-producing plan or the artifacts have expired, the artifact-consuming job will fail.

---

# Artifact HTTP compression

Disable HTTP compression for artifact transfers

```yaml
---
version: 2
# ...
  artifacts:
    -
      name: Special Reports
      location: target/reports
      pattern: 'special/*.xml'
      shared: true
      required: true
      httpCompressionOn: false
```

You can disable HTTP compression during artifact transfers by setting the `httpCompressionOn` property as `false`.

# Using a shared artifact in another job of the same plan

Create a WAR and ZIP artifacts in the Build stage and use WAR artifact in the Test stage.

```yaml
---
version: 2
# ...
Build WAR:
  artifacts:
    -
      name: WAR
      location: target
      pattern: '*.war'
      shared: true
    - name: zip
      location: target
      pattern: '*.zip'
      shared: true

Test app:
  artifact-subscriptions:
    -
      artifact: WAR
      destination: deploy
```

You can share artifacts between jobs in **different stages** using artifact dependencies. For example, you may want to run acceptance tests on a build, sharing the same WAR from one job to another without rebuilding it each time. Each time the artifact is shared with a subsequent job, it is copied to the job's agent.

In YAML Specs, every shared artifact will be automatically added as a dependency to the subsequent stages by default. It's possible to change this behavior as presented in the sample code.

You can refer only to artifacts from jobs in previous stages that have been marked as shared. Destination directory is relative to the build directory. Don't use the absolute path to refer to the destination directory.

# Using a shared artifact in another plan

Download all artifacts from another plan.

```yaml
---
version: 2
# ...
  tasks:
    - artifact-download:
        source-plan: PROJECTKEY-PLANKEY
```

Download selected artifacts from another plan.

```yaml
---
version: 2
# ...
  tasks:
    - artifact-download:
        source-plan: PROJECTKEY-PLANKEY
        artifacts:
          - name: WAR
          - name: DATA
```

You have to use the Artifact Downloader Task for this purpose. In this task point to the build plan that is the source of the artifact(s) you need to download. You can either download all artifacts from the plan or selected ones as well as specify target folder for them.

# Notifications

Defining notification for plan.

```yaml
---
version: 2
# ...
notifications:
  - recipients:
      - users:
          - admin
      - emails:
          - admin@example.com
      - groups:
          - admins
      - responsible
      - watchers
      - committers
    events:
      - plan-failed
      - plan-completed
      - plan-status-changed
      - plan-comment-added
      - plan-responsibility-changed
      - job-error
      - job-completed
      - job-status-changed
      - job-failed
      - job-first-failed
      - job-hung
      - job-queue-timeout
      - job-queued-without-capable-agents
  - recipients:
      - committers
    events:
      - plan-failed:
          failures: 3
      - job-error:
          first-only: false
  - recipients:
      - webhook:
          name: "Build webhook"
          url: "http://localhost:8080"
    events:
      - plan-completed
```

Defining notification for deployment project.

```yaml
---
version: 2
# ...
Environment:
  notifications:
    - recipients:
        - users:
          - admin
          - joe
        - emails:
          - admin@gmail.com
          - joe@gmai.com
```

```
      events:
        - deployment-failed
        - deployment-finished
        - deployment-started-and-finished
    - events: deployment-failed
      recipients:
        - groups:
          - bamboo-admin
          - bamboo-users
```

Notifications in Bamboo are triggered by a range of events involving a plan and its jobs, including build completion, build outcomes and comments being posted against build results. You can configure whether notifications are sent for a particular event for each plan and job, and who they are sent to.

To learn more about Bamboo plan notifications, read Bamboo notifications documentation.

# Miscellaneous plugins

Miscellaneous plugins configuration on a plan level. Concurrent builds plugin is configured with a dedicated builder, while Build expiry and Artifact handler are using `AllOtherPluginsConfiguration` or `all-other-apps` tag.

```
version: 2
#...
other:
  concurrent-build-plugin: 5
  all-other-apps:
    custom:
      artifactHandlers:
        useCustomArtifactHandlers: false
      buildExpiryConfig:
        duration: 5
        enabled: true
        expiryTypeResult: true
        maximumBuildsToKeep: 1
        period: days
```

Miscellaneous plugins configuration on a job level. Regex Pattern Labeling, Build Hanging Config, Clover, and CleanWorkingDir plugins are configured with `all-other-apps` tag.

```
version: 2
# ...
Default job:
  tasks: []
  other:
    clean-working-dir: true
    all-other-apps:
      custom:
        auto:
          regex: (PERFORMANCE_IMPROVED|PERFORMANCE_DETERIORATED)
          label: \1
        clover:
          path: results
          integration: custom
          exists: true
          useLocalLicenseKey: true
        buildHangingConfig:
          minutesBetweenLogs: '7'
          minutesQueueTimeout: '25'
          multiplier: '2.6'
          enabled: 'true'```
```

Miscellaneous plugins are used for various additional functionalities for plans and jobs like Build expiry and Clover code coverage. You can configure these plugins from the *Other* tab in the web interface.

Some miscellaneous plugins work only on the job level while other work only on the plan level. When you choose a plugin, make sure you're configuring it on the right level.

In case a plugin doesn't have a dedicated builder, you can use `AllOtherPluginsConfiguration` to provide configuration for such plugins as a workaround. Refer to plugin documentation to obtain a list of valid keys. Note that keys imported with `AllOtherPluginsConfiguration` or `all-other-apps` are not validated.

# Deployment projects

```yaml
---
version: 2

deployment:
  name: Deploy satellites
  source-plan: ROCKET-LAUNCH

release-naming: release-1.1
```

A deployment project in Bamboo is a container for holding the software project you are deploying: releases that have been built and tested, and the environments to which releases are deployed.

To learn more about Bamboo deployment projects, see Bamboo documentation.

## Release naming

> Specifying release name that is automatically incremented when release is created.
>
> The subsequent release names will be named "release-1.2", "release-1.3" and so on.

```yaml
---
version: 2
release-naming:
  next-version-name: release-1.1
  applies-to-branches: true
  auto-increment: true
```

```yaml
---
version: 2
release-naming:
  next-version-name: release-${bamboo.release-number}
  auto-increment-variables:
    - release-number
```

```yaml
---
version: 2
release-naming: release-1
```

You can define how releases should be named when they are created by Bamboo. You may decide if this naming scheme should be applied to releases created from branch builds. If not, those releases are named after the branch in the format:

`<branch_name>-<build_number>`

# Environments

An environment represents the servers or groups of servers where the software release has been deployed to, and the tasks that are needed for the deployment to work smoothly. You can also define when and to whom notifications should be send.

Configuring a deployment environment.

```yaml
---
version: 2
# ...
environments:
  - Test
  - QA
  - Prod

Test:
  tasks:
    - clean
    - artifact-download:
        destination: /workdir
```

## Execution prerequisites

You can configure the environment to require that certain prerequisites are met before a release can be deployed to it.

The release approval prerequisite specifies whether the release needs approval to be deployed to the given environment.

The possible values are:

| | |
|---|---|
| **not-broken** | The release can be deployed to the environment if it's not marked as broken by anyone. This is the default value. |
| **approved** | The release can be deployed to the environment if there is at least one approval and the release is not marked as broken by anyone. |
| **none** | Any release can be deployed to the environment. |

Configuring a deployment environment to require approved release.

```yaml
---
version: 2
# ...
QA:
  release-approval-prerequisite: approved
  # ...
```

## Tasks

In a manner similar to Plans, the process of deploying to an environment is split into tasks, such as downloading the build artifacts or running a script.

To learn more about configuring tasks in Bamboo Specs, see Tasks. They share the same configuration details.

## Variables

Defining deployment environment variables.

```
---
version: 2
# ...
QA:
  variables:
    variableName: variableValue
```

You can define variables specific to a deployment environment.

To learn more about Bamboo deployment variables, see Bamboo variables documentation.

In case of storing passwords in variables, you can use the Bamboo Specs Encryption and store the encrypted data in the repository.

# Deployment triggers

Triggering in Bamboo allows deployments to a specific environments to be started automatically. The following triggering methods can be used with deployment environments:

## After successful plan trigger

Execute deployment when linked plan build is complete.

```
---
version: 2
# ...
Environment:
  triggers:
    - build-success
```

```
---
version: 2
triggers:
  - build-success:
      branch: stage-ready-branch #if environment should be updated on branch build
```

## After successful stage trigger

Execute deployment when linked plan's stage build is complete.

```yaml
---
version: 2
triggers:
  - stage-success:
      stage: Space stage
```

```yaml
---
version: 2
Environment:
  triggers:
    - stage-success: Test
    - stage-success:
        stage: Integration tests
        branch: integration-branch
```

## After successful deployment trigger

Execute deployment when deployment to another environment is complete.

```yaml
---
version: 2
Environment:
  triggers:
    - environment-success: Staging
```

## Scheduled trigger

Execute deployment by schedule.

```yaml
---
version: 2
Environment:
  triggers:
    - cron:
        expression: 0 0 0 ? * *
        artifact-branch: your-artifact-branch
```

# Dependencies

Dependencies are configured with setting options on `dependencies` tag.

```yaml
---
version: 2
# ...
dependencies:
  require-all-stages-passing: true
  plans:
    - PROA-PLAN2
    - PROA-PLAN3
```

A dependency blocking strategy which block a trigger if a parent's build is in progress. It does not apply for branch plans.

```yaml
---
version: 2
# ...
dependencies:
  enabled-for-branches: false
  block-strategy: block_if_parent_in_progress
```

In Bamboo, you can define build dependencies, which will for example trigger a plan build when another plan's build has successfully completed.

Both in Bamboo and in Bamboo Specs it is possible to define dependency blocking strategies, such as:

| | |
|---|---|
| **Don't block** | When triggered by a source code update, the plan will always be built, regardless of any parent plan build dependencies. |
| **Block build if parent builds are queued or in progress** | When triggered by a source code update, the plan will not be built if its parent plans are building or are waiting in the build queue. |
| **Block build if parent plans have unbuilt changes** | When triggered by a source code update, the plan will not be built if its parent plans are building, are waiting in the build queue, or have changes. When Bamboo finds parent plans with source repository changes, those plans will be triggered and your plan will be blocked. |

Child plans are triggered after execution of a parent plan. You can configure child plans from the *Dependencies* tab in the web interface.

Bamboo Specs override existing settings. Omitting the dependencies section results in removing of all existing dependencies.

Read more at:

- Setting up plan build dependencies
- Dependency blocking strategies

Automatic dependency management is controlled in the web interface from the *Dependencies* tab. However, in Bamboo Specs, this setting is configured using a plugin method (see examples). Automatic dependencies are omitted when exporting to Bamboo Specs. In case a plan already has an automatic dependency on another plan, an attempt to add the same dependency manually will be ignored.

# Project permissions

It is currently not possible to create project permissions with YAML Specs.

Project permissions allow a user to manage project and plan permissions. These are the same permissions as the ones accessible from the project configuration page.

Permissions for a project and its plans can be set for:

- specific users
- specific groups
- logged in users
- anonymous users

## Project Permissions

Project permissions include viewing, creating plans, and administering a project.

| | |
|---|---|
| **View** | View the project, its plans, and plan builds. |
| **Create** | Create a plan in the project. |
| **Admin** | Edit all aspects of the project including permissions and resources. |

This feature is available for Bamboo DC licenses only.

Project permissions include creating project repositories.

| | |
|---|---|
| **Create Repository** | Create a project repository. |

## Project Plan Permissions

Project plan permissions are inherited by all the plans within the specified project.

| | |
|---|---|
| **View** | View the plan and its builds. |
| **Edit** | View and edit the configuration of the plan and its jobs, not including permissions or stages. |
| **Build** | Trigger a manual build, or suspend and resume the plan. |
| **Clone** | Clone the plan. |
| **Admin** | Edit all aspects of the plan including permissions and stages. |

This feature is available for Bamboo DC licenses only.

| | |
|---|---|
| **View Configuration** | View the plan, its builds and configuration. |

## RSS Permissions

Repositories can be allowed to update a specific project using RSS. This applies to global-level repositories only.

# Plan Permissions

```
---
version: 2
```

```yaml
plan: PLAN-KEY

plan-permissions:
  - users: admin
    groups: bamboo-admins
    roles: logged-in
    permissions: view
```

Plan permissions allow a user to control access to the functions of the build plan. These include viewing, editing, building, cloning and administering a build plan. These are the same permissions which can be accessed from the build plan configuration page.

Plan permissions can be set for:

- specific users
- specific groups
- logged in users
- anonymous users

| | |
|---|---|
| **View** | View the plan and its builds. |
| **Edit** | View and edit the configuration of the plan and its jobs, not including permissions or stages. |
| **Build** | Trigger a manual build, or suspend and resume the plan. |
| **Clone** | Clone the plan. |
| **Admin** | Edit all aspects of the plan including permissions and stages. |

This feature is available for Bamboo DC licenses only.

| | |
|---|---|
| **View Configuration** | View the plan, its builds, and configuration. |

Sets the plan permissions to defaults:

| | |
|---|---|
| **VIEW** | For *logged in users*. |
| **VIEW** | For *anonymous users*. |

## Publishing

In Java Specs, plan permission is a top level Bamboo Spec entity (like Plan or Deployment) and needs to be published to the Bamboo instance.

# Deployment Permissions

In Java Specs, deployment and environment permission is a top level Bamboo Specs entity (like Plan or Deployment) and needs to be published to the bamboo instance.

In YAML Specs, deployment and environment permissions must be specified in a separate YAML document.

Configuring deployment permissions. Note that the permissions object has to be published to the Bamboo server.

```yaml
---
version: 2
deployment: My deployment

deployment-permissions:
  - users: admin
    groups: bamboo-admin
    roles: anonymous
    permissions:
      - view
```

Configuring environment specific permissions. Note that the permissions object has to be published to the Bamboo server.

```yaml
---
version: 2
deployment: My deployment

# These permissions apply to the deployment project
deployment-permissions:
  - users:
      - admin

# These permissions apply to all environments defined in this deployment project
default-environment-permissions:
  - users:
      - admin
    groups:
      - bamboo-remote-users
    permissions:
      - view
      - edit
      - deploy

# Permissions specific to an environment
environment-permissions:
  - QA:
      - users:
          - admin
        groups:
          - bamboo-remote-users
        permissions:
          - view
          - edit
          - deploy
      - roles:
          - logged-in
          - anonymous
        permissions:
          - view
```

# Java Specs Utilities

The Bamboo Specs library comes with several helper classes, which can ease task of deploying a spec to Bamboo.

- BambooSpec - an annotation to mark classes which can publish a plan; used by the Bamboo Specs Runner plugin for Maven
- BambooServer - publish a plan, a plan permission or a deployment project to Bamboo running on certain URL using a provided credentials

- UserPasswordCredentials - an interface for providing credentials, you can implement it to provide custom source
- FileUserPasswordCredentials - read login and password from '.credentials' file from current working directory
- SimpleUserPasswordCredentials - pass login and password directly
- FileTokenCredentials - read personal access token from '.credentials' file
- SimpleTokenCredentials - pass personal access token directly
- RssRuntimeContext - when executed by RSS feature, provides information about RSS execution context, such as current branch or server name

# Multiple Bamboo servers

It is sometimes convenient to use several Bamboo instances while keeping configuration of plans/deployments in the same repository. You can use the helper method from RssRuntimeContext to execute your code in a specific Bamboo instance. YAML allows you to use regular expressions with names of Bamboo instances. If there's match for the instance name, YAML document is applied.

```yaml
---
version: 2
server-name: my_bamboo
# ...

QA:
  tasks: []
```

# Understanding YAML in Specs

## Starting with YAML

If you are new to YAML, we recommend that you read the basics of YAML before starting with YAML Specs.

Also, if you are looking how to create your first `bamboo.yaml` definition, you can read a basic tutorial in our docs.

## YAML Localization

Sample of a repository with YAML Specs and source code

```
/my-yaml-specs-repository
├── bamboo-specs
│   ├── bamboo.yaml
│   ├── planA.yaml
│   └── planB.yaml
├── src
│   └── ...
└── README.md
```

Every YAML Specs file should be inside the `bamboo-specs` folder, on the root of your repository.

The `bamboo.yaml` is the root file for all your definitions.

---

# Documents

You can have as many documents as you need

```
---
# Plan definition
version: 2
# ...

---
# Second plan definition
version: 2
# ...

---
# ...
```

YAML is divided into documents. Documents are sections inside the same file which contains your YAML Specs definitions. The same file can contain multiple documents.

Every YAML document starts with:

`---`

Bamboo Specs will evaluate each document as a self-contained piece. Every anchor, include and key used in a document are local to it.

#### FORMATS

Two valid documents inside the same file

```
---
version: 2
plan:
    project-key: MARS
    key: THESHIP

---
version: 2
deployment:
    name: Deploy the ships
```

A separated file with permissions for the plan and deployments

```
---
version: 2
plan:
  key: PLAN-KEY

plan-permissions:
    # ...
---
version: 2
deployment:
  name: My deployment
```

```
deployment-permissions:
    # ...
```

Bamboo Specs accepts the following definitions of documents format:

- plans
- deployment projects
- plan permissions
- deployment permissions

Bamboo understands each of these formats in separated documents.

You can organize your repositories to contain different combinations of YAML Specs. For example, you could have one repository hosting your source code together with YAML Specs, another repository with only the YAML Specs for deployments of this code, and a third repository to keep only the permissions. With this scenario, you could have more control of who would be able to change different parts of your definitions in Specs, being more restrictive for permissions, but allowing all developers in the company to access the plans.

---

# Includes

Including a second plan into YAML Specs

```
---
!include 'second-plan.yml'

---
# Another document with a different definition format
version: 2
# ...
```

Using your main YAML file to include every other plan

```
# bamboo.yaml
---
!include 'planA.yaml'
---
!include 'planB.yaml'


# planA.yaml
---
version: 2
# ...


# planB.yaml
---
version: 2
# ...
```

Reusing smaller pieces like tasks or environments definitions, across multiple definitions.

```
# bamboo.yaml
---
version: 2
plan:
  key: BUILD-WEB
# ...
Build job: !include 'builds/build-job.yaml'
```

```yaml
---
version: 2
plan:
  key: BUILD-COR
# ...
Build: !include 'builds/build-job.yaml'



# builds/build-job.yaml
tasks:
  -
    script:
      interpreter: /bin/sh
      scripts:
        - echo 'npm run start' > script.sh
        - echo 'Built' > output.log
```

YAML Specs extends YAML with the `!include 'file'` tag. The include tag expands to the content of the file where it is placed.

Includes can be used in different ways:

- split your definitions into different files
- reuse separated pieces of configuration to build different definitions
- define variables or other single values

The include tag looks in the following way:

`!include 'filepath'`

The `filepath` is relative to the `/bamboo-specs` folder, and it cannot point to files outside of this folder. Also, only `.yaml` or `.yml` files are accepted.

This tag is flexible, and it can be used anywhere in the file, as long as it follows the YAML format.

If you use `!include` tag to reference full plan or deployment make sure it is the only tag at a document, e.g.

Correct

`!include 'planA.yaml'`

Incorrect

`version: 2`

`!include 'planA.yaml'`

---

# Variables (Anchors)

Anchors accept any YAML type

```yaml
---
version: 2

plan: &plan-map
  project-key: MARS
  key: P11P

stages:
  -
    Primary build stage: &jobs-set
      - Build pi
      - Test pi
      - Run pi
```

```
  -
    Second pi phase: *jobs-set
```

In YAML, you can define anchors which work like variables:

`key: &plan-key ROCKET`

Later in the same document, you would be able to reference the plan key by using:

`*plan-key`

---

# Merging keys

The merge tag works for sequences and maps

```
my-map: &map
  anchored-map: I will merge to the parent
  maybe-map: I will maybe merge to the parent

# This map will contain both `my-map` keys
second-map:
  <<: *map

# This map overrides the `maybe-map` key
third-map:
  <<: *map
  maybe-map: This is not my-map value
```

```
Build Ship Server: &build-ship
  tasks:
    -
      script: |
        echo 'Building the ship...'

Second Ship:
  <<: *build-ship
  final-tasks:
    - test-parser: testng
```

The merge key in YAML `<<` will always merge the content of its child to the parent.

Merge keys are specially useful with YAML anchors.

---

# YAML Tips

Some tips on using YAML Bamboo Specs.

#### MULTI-LINE STRINGS

YAML allows multiple-line strings

```
---
version: 2
#...
  script: |
    #!/bin/node
```

```
    const io = require('io')
    console.log('hello nodejs')
```

**SEQUENCES FORMATS**

```yaml
---
version: 2
# ...
stages:
  - Build binaries:
      - Build binaries
  - Run tests:
      manual: false
      final: false
      jobs:
        - Unit tests
        - Integration tests
  - Deploy:
      manual: true
      jobs:
        - Deploy
```

Leaving the sequence separators in different lines might help splitting

```yaml
---
version: 2
# ...
stages:
  -
    Build binaries:
      - Build binaries
  -
    Run tests:
      manual: false
      final: false
      jobs:
        - Unit tests
        - Integration tests
  -
    Deploy:
      manual: true
      jobs:
        - Deploy
```

You can also use JSON-like syntax and avoid deep nesting

```yaml
---
version: 2
# ...
stages:
  -
    Build binaries: [Build binaries]
  -
    Run tests:
      manual: false
      final: false
      jobs: [Unit tests, Integration tests]
  -
    Deploy:
      manual: true
      jobs: [Deploy]
```

YAML Specs makes extensive usage of mapped sequences. On the sidebar, you can see some examples of format alternatives to write mapped sequences.

# YAML Specs Reference

Complex plan definition:

```yaml
---
version: 2
server-name: team_.*
plan:
  project-key: PROJ
  key: PLAN
  name: My Plan

docker:
  image: oracle
  volumes:
    /home/user: /home/user
    /opt: /opt
  use-default-volumes: false
  docker-run-arguments:
    - --net=host

repositories:
  - linked-repository1 # only if name is unique for linked and project repositories
  - linked repository 2:
      scope: global
  - project repository 1 # only if name is unique for linked and project repositories
  - project repository 2:
      scope: project
  - plan repository 1:
      type: git
      url: ssh://git@bitbucket.org:my-company/my-repository.git
      branch: master
      shared-credentials: identifier
  - plan repository 2:
      type: git
      url: https://bitbucket.org/my-company/my-repository
      branch: master
      shared-credentials:
        name: BBC Token
        scope: project
      command-timeout-minutes: 180
      lfs: false
      verbose-logs: false
      use-shallow-clones: false
      cache-on-agents: true
      submodules: false
      fetch-all: false
      change-detection:
        quiet-period:
          quiet-period-seconds: 60
          max-retries: 5
        exclude-changeset-pattern: .*draft.*
        file-filter-type: include_only
        file-filter-pattern: .*\.java
  - plan repository 3:
      type: git
      url: ssh://git@bitbucket.org:my-company/my-repository.git
      branch: master
      ssh-key: ENCRYPTED_KEY
      ssh-key-passphrase: ENCRYPTED_PASSPHRASE
  - bitbucket-cloud:
      type: bitbucket
      slug: atlassian/bamboo-specs
      branch: master
      command-timeout-minutes: 180
      use-shallow-clones: false
      cache-on-agents: false
```

```yaml
      submodules: false
      verbose-logs: false
      fetch-all: false
      lfs: false
      viewer: com.atlassian.bamboo.plugins.atlassian-bamboo-plugin-bitbucket:bbCloudViewer
  - subversion:
      plugin-key: com.atlassian.bamboo.plugin.system.repository:svnv2
      server-config:
        repository.svn.useExternals: false
        repository.svn.tag.create.autodetectPath: true
        repository.svn.authType: password
        repository.svn.branch.create.autodetectPath: true
        repository.svn.userPassword: ENCRYPTED_PASSWORD
        repository.svn.useExport: false
        repository.svn.repositoryRoot: http://localhost/my/svn
      branch-config:
        repository.svn.branch.displayName: svn
  - Github:
      type: github
      repository: atlassian/bazel
      branch: master
      user: atlassian
      password: ENCRYPTED_PASSWORD
      command-timeout-minutes: 180
      lfs: false
      verbose-logs: false
      use-shallow-clones: false
      cache-on-agents: true
      submodules: false
      fetch-all: false
      viewer: com.atlassian.bamboo.plugins.atlassian-bamboo-plugin-git:githubViewer
  - Bitbucket Server:
      type: bitbucket-server
      server: bitbucketServerApplink
      project: BBSPROJECT
      slug: my-repository-slug
      clone-url: ssh://my.bitbucket.server:7999/BSSPROJECT/my-bitbucket-repository.git
      public-key: PUBLIC KEY
      private-key: ENCRYPTED_PRIVATE_KEY
      branch: master
      lfs: true
      use-shallow-clones: true
      submodules: false
      command-timeout-minutes: 30
      fetch-all: false
      viewer: com.atlassian.bamboo.plugins.stash.atlassian-bamboo-plugin-stash:bbServerViewer

triggers:
  - polling: 130
  - polling:
      period: 150
  - polling:
      cron: 0 0/30 9-19 ? * MON-FRI
      repositories:
        - bitbucket-cloud
      conditions:
        - green-plan:
          - PROJECTKEY-PLANKEY
        - all-other-conditions:
            custom.rejectBranchBuildWithoutChange.enabled: true
  - cron: 0 * * * ? *
  - cron:
      expression: 0 0 * * ? *
  - remote
  - remote: 192.168.0.1
  - remote:
      ip: 192.168.0.2

notifications:
  - recipients:
      - users:
          - admin
```

```yaml
      - emails:
          - admin@example.com
    events:
      - plan-failed
      - job-error
  - recipients:
      - responsible
      - watchers
    events:
      - plan-failed: 3
      - job-error:
          first-only: false
  - recipients:
      - committers
    events:
      - plan-failed:
          failures: 2
      - plan-completed
      - plan-status-changed
      - plan-comment-added
      - plan-responsibility-changed
      - job-completed
      - job-status-changed
      - job-failed
      - job-first-failed
      - job-hung
      - job-queue-timeout
      - job-queued-without-capable-agents

variables:
  password: admin
  username: admin

branches:
  create: for-pull-request
  delete:
    after-deleted-days: 40
    after-inactive-days: 10
  integration:
    merge-from: master
    push-on-success: true
  link-to-jira: false

dependencies:
  require-all-stages-passing: true
  enabled-for-branches: false
  block-strategy: block_if_parent_in_progress
  plans:
    - PROA-PLAN2
    - PROA-PLAN3

branch-overrides:
  -
    integration-branch:
      stages:
        - Build binaries:
            - Build binaries
        - Run tests:
            manual: false
            final: false
            jobs:
              - Unit tests
              - Integration tests
        - Deploy:
            manual: true
            jobs:
              - Deploy
        - Cleanup:
            final: true
            jobs:
              - Cleanup
  -
```

```yaml
    dev-new-feature:
      stages:
        - Build binaries:
            - Build binaries
        - Cleanup:
            final: true
            jobs:
              - Cleanup
  -
    dev-.*:
      stages:
        - Build binaries:
            - Build binaries
        - Run tests:
            manual: false
            final: false
            jobs:
              - Unit tests
              - Integration tests
        - Cleanup:
            final: true
            jobs:
              - Cleanup

other:
  concurrent-build-plugin: 5
  all-other-apps:
    custom:
      artifactHandlers:
        useCustomArtifactHandlers: false
      buildExpiryConfig:
        duration: 5
        enabled: true
        expiryTypeResult: true
        maximumBuildsToKeep: 1
        period: days

stages:
  - Build binaries:
      - Build binaries
  - Run tests:
      manual: false
      final: false
      jobs:
        - Unit tests
        - Integration tests
  - Deploy:
      manual: true
      jobs:
        - Deploy
  - Cleanup:
      final: true
      jobs:
        - Cleanup

Build binaries:
  key: BB
  tasks:
    - script:
        interpreter: /bin/sh
        scripts:
          - echo 'echo success' > script.sh
          - echo 'success' > output.log
    - maven:
        executable: Maven 3
        jdk: JDK 1.8
        goal: ${bamboo.maven.tasks}
        tests: '**/target/test-reports/*.xml'
        environment: MAVEN_OPTS="-Xmx1024m"
        working-dir: sub-dir
        project-file: pom-file.xml
        use-return-code: true
```

```yaml
    - inject-variables:
        file: folder\file.txt
        scope: RESULT # case insensitive
        namespace: myspace
        conditions:
          - variable:
              equals:
                planRepository.branch: development
    - any-task:
        plugin-key: com.atlassian.bamboo.plugins.variable.updater.variable-updater-generic:variable-updater
        configuration:
          variable: bamboo.variable.name
          strategy: DEPLOYMENT
          variableScope: JOB
  artifacts:
    - name: Binaries
      location: .
      pattern: script.sh
      required: true
      shared: true
    - name: Logs
      pattern: "**/*.log"
      required: false
      shared: false
    - name: All
      pattern: "**/*"
  requirements:
    - hasDocker
  docker:
    image: ubuntu
    use-default-volumes: true
  other:
    clean-working-dir: true
    all-other-apps:
      custom:
        clover:
          path: results
          integration: custom
          exists: true
          useLocalLicenseKey: true

Unit tests:
  key: UT
  tasks:
    - script:
        scripts:
          - touch report.xml
  final-tasks:
    - test-parser:
        type: junit
        test-results: report.xml
        ignore-time: true
  artifact-subscriptions: []

Integration tests:
  key: IT
  tasks:
    - clean
    - script:
        - touch report.xml
  final-tasks:
    - test-parser: testng
    - test-parser:
        type: mocha
        test-results:
          - mocha-1.json
          - mocha-2.json
    - test-parser:
        type: mstest
        test-results:
          - tests1\results.trx
          - tests2\results.trx
```

```
    - inject-variables: tests.txt
  docker: postgres
  artifact-subscriptions:
    -
      artifact: Binaries
      destination: bin

Deploy:
  tasks: []
  final-tasks: []
  artifacts: []
  requirements: []
  other: {}

Cleanup:
  key: CLEAN
```

Complex deployment project:

```
---
version: 2
server-name: deployment.*
deployment:
  name: Deployment project name
  source-plan: PLAN-KEY

release-naming:
  next-version-name: 0.${bamboo.buildNumber}.${bamboo.quickCompileMavenGoals.experimental}
  applies-to-branches: true
  auto-increment: true
  auto-increment-variables:
    - quickCompileMavenGoals.experimental

environments:
  - QA
  - Staging
  - Production

QA:
  tasks:
    - clean
    - artifact-download:
        destination: /workdir
    - script:
        - echo 'hello world'
        - echo "I'm done"
    - artifact-download:
        artifacts:
          - name: my-artifacts
            destination: /my-artifacts
  final-tasks:
    - script:
        - echo "let's clean it up"
    - artifact-download:
        artifacts:
          - name: my-artifacts
            destination: /my-artifacts
          - name: their-artifacts
  requirements:
    - isDragonLazy
  docker:
    image: docker-image-name
  variables:
    variableName: variableValue
  triggers:
    - build-success
    - stage-success:
        stage: Stage 2
```

```yaml
    - stage-success:
        stage: Stage 3
        branch: develop-branch
  notifications:
    - events:
        - deployment-started-and-finished
      recipients:
        - users:
          - admin
          - joe
        - groups:
          - bamboo-admin
    - events:
        - deployment-failed
        - deployment-finished
      recipients:
        - emails:
          - admin@gmail.com
          - joe@gmai.com

Staging:
  tasks:
    - clean
    - artifact-download:
        destination: /workdir
    - script:
        - echo 'hello world'
        - echo "I’m done"
    - artifact-download
    - any-task:
        plugin-key: com.atlassian.bamboo.plugins.variable.updater.variable-updater-generic:variable-updater
        configuration:
          variable: bamboo.variable.name
          strategy: DEPLOYMENT
          variableScope: JOB
  docker:
    image: docker-image-name
    volumes:
      ${bamboo.working.directory}: ${bamboo.working.directory}
      ${bamboo.tmp.directory}: ${bamboo.tmp.directory}
    use-default-volumes: false
    docker-run-arguments:
      - --net=host
  variables:
    variableName: variableValue
  triggers:
    - stage-success: dragons stage
    - build-success:
        branch: stage-ready-branch
  notifications:
    - recipients:
        - users:
          - admin
          - joe
        - groups:
          - bamboo-admin
          - bamboo-users
        - emails:
          - admin@gmail.com
          - joe@gmai.com
      events:
        - deployment-failed
        - deployment-finished
        - deployment-started-and-finished

Production:
  tasks:
    - clean
    - artifact-download:
        name: AllPackages
        destination: /workdir
    - script:
```

```
        interpreter: cmd
        scripts:
          - echo 'hello world'
          - echo "I’m done"
  variables:
    variableName: variableValue
  triggers:
    - environment-success: Staging
    - cron: 0 0 0 ? * *
  notifications:
    - events: deployment-failed
      recipients:
        - groups:
          - bamboo-admin
          - bamboo-users
        - groups:
          - bamboo-devs
```

Plan permissions:

```
---
version: 2
server-name: team_.*
plan:
  key: PLAN-KEY

plan-permissions:
  - users:
      - admin
      - non-admin
    groups:
      - bamboo-admins
      - bamboo-users
    roles:
      - anonymous
      - logged-in
    permissions:
      - view
  - users:
      - some user
      - another user
    groups:
      - bamboo-power-admins
      - bamboo-power-users
    permissions:
        - view
        - edit
        - build
        - clone
        - admin
```

Deployment project permissions:

```
---
version: 2
server-name: deployment.*
deployment:
  name: My deployment

deployment-permissions:
  - users:
      - local-admin
      - remote-admin
    groups:
      - bamboo-remote-users
    permissions:
      - view
```

```yaml
      - edit
  - users:
      - joe
    groups:
      - bamboo-users
    permissions:
      - view

default-environment-permissions:
  - users:
      - local-admin
      - remote-admin
    groups:
      - bamboo-remote-users
    permissions:
      - view
      - edit
      - deploy

environment-permissions:
  - QA:
      - users:
          - local-admin
          - remote-admin
        groups:
          - bamboo-remote-users
        permissions:
          - view
          - edit
          - deploy
      - roles:
          - logged-in
          - anonymous
        permissions:
          - view
  - Production:
      - groups:
          - bamboo-power-users
        permissions:
          - view
          - edit
      - roles:
          - logged-in
        permissions:
          - view
          - deploy
```

View cookie preferences